

FACULTY
OF
INFORMATION
AND
COMMUNICATION TECHNOLOGY

NATIONAL DIPLOMA: COMPUTER SYSTEMS ENGINEERING

FINAL PROJECT REPORT

SUBJECT NAME: Project Design 376

SUBJECT CODE: PD2316D

PROJECT TITLE: Smart Registration System for Exam Venue Access Based on Face Recognition

LECTURER NAME: Muwanguzi Mark Ntume and Chunling Du

STUDENTS NAME: TC Mance

STUDENTS NUMBER: 222138469

EMAIL ADDRESS: 222138469@tut4life.ac.za

TEAM MEMBERS: None (individual project)

SUBMISSION DATE: 5 November 2026

TABLE OF CONTENTS

List of all tables and figures....3
List of Figures....3
List of Tables....3
List of Abbreviations....4
ABSTRACT....5
1. INTRODUCTION....5
1.1 Problem statement....5
1.2 Motivation....5
1.3 Why the problem is not yet solved and why it is difficult....5
1.4 Approach and assumptions....6
1.5 Main results and contributions....6
1.6 Organisation of this report....6
2. BACKGROUND....6
2.1 Face detection and face recognition....6
2.2 Measuring a biometric system....7
2.3 Ultrasonic distance gating....7
2.4 Web technologies....7
3. PAST/RELATED WORK....7
4. PROJECT MANAGEMENT....8
4.1 Strategy and task breakdown....8
4.2 Timeline....8
4.3 Changes from the original plan....9
5. TECHNICAL SECTIONS....9
5.1 System architecture....9
5.2 Hardware design: the door unit....12
5.2.1 Door unit behaviour (srs_door.ino)....13
5.2.2 Design calculations....14
5.3 Door software: the PC door program....14
5.3.1 Access verification....14
5.3.2 Enrolment....16
5.3.3 Working offline....17
5.4 Backend REST API and database....17
5.4.1 Business rules....19
5.4.2 Security and privacy....21
5.5 Web console (SRS Console)....21
6. PROJECT RESULTS AND ANALYSIS....24
6.1 Test strategy....24
6.2 Recognition benchmark (FAR and FRR)....25
6.2.1 Analysis....26
6.3 Speed....27
6.4 Achievement of the objectives....27
6.5 Functional tests of the complete system....27
7. CHALLENGES FACED....28
8. FINAL BUDGET....28
9. PROJECT DELIVERABLES....29
10. RECOMMENDATIONS AND FUTURE WORK....29
11. CONCLUSIONS....30

12. REFERENCES 30
APPENDICES 32
Appendix A: Door unit serial protocol 32
Appendix B: Device API used by the door program 32
Appendix C: On-site accuracy test procedure 32
Appendix D: Benchmark method 32
Appendix E: Source code 33

List of all tables and figures

List of Figures

Figure 1: Planned (proposal) versus actual project timeline 9
Figure 2: System architecture of the final Smart Registration System 10
Figure 3: UML use case diagram 11
Figure 4: UML deployment diagram 12
Figure 5: Door unit block and wiring diagram 13
Figure 6: UML state machine of the Arduino door unit (srs_door.ino) 14
Figure 7: UML activity diagram: access verification, in swimlanes per part 15
Figure 8: UML sequence diagram: one access verification 16
Figure 9: UML sequence diagram: student enrolment 17
Figure 10: UML class diagram of the database (11 tables) 19
Figure 11: Exam sessions, once-per-session entry and automatic sign-out 20
Figure 12: UML sequence diagram: booking an exam, e-mails and automatic sign-out 21
Figure 13: Sign-in screen 22
Figure 14: Dashboard: live statistics, students recognised now and exam hall bookings 22
Figure 15: Student enrolment with consent confirmation 23
Figure 16: Venues and devices 23
Figure 17: Exam hall bookings 24
Figure 18: Exam calendar 24
Figure 19: Similarity scores of same-person and different-person pairs 26
Figure 20: FAR and FRR against the match threshold 26

List of Tables

Table 1: Abbreviations used in this report 4
Table 2: Comparison of the SRS with existing face-recognition access products 7
Table 3: Work breakdown and status 8
Table 4: Changes from the proposal and design document 9
Table 5: Door unit wiring (Arduino Mega 2560) 13
Table 6: Backend API route groups 17
Table 7: Automated backend tests (node:test) – run on 1 November 2026 25
Table 8: 1:1 verification results for different thresholds (LFW, 6 000 pairs) 25
Table 9: 1:N door simulation (100 enrolled students, mean ± standard deviation over 5 runs). 26
Table 10: Proposal objectives and results 27
Table 11: Door unit functional test (Test hardware window and end-to-end checks) 27
Table 12: Final budget (estimated South African prices) 28
Table 13: Planned and delivered outcomes 29
Table 14: Serial messages between the Arduino and the PC (9600 baud, one per line) 32
Table 15: Device API endpoints (header X-Device-Key on every call) 32

List of Abbreviations

Table 1: Abbreviations used in this report

Term	Meaning
API	Application Programming Interface: the web addresses one program offers to another
CSV	Comma-Separated Values: a plain spreadsheet file
EER	Equal Error Rate: the threshold point where FAR equals FRR
Embedding	A list of 128 numbers that represents a face so two faces can be compared
FAR / FRR	False Acceptance Rate (a stranger is let in) / False Rejection Rate (an enrolled student is refused)
HTTPS	Secure (encrypted) version of HTTP, the request/response protocol of the web
I²C	Inter-Integrated Circuit: a two-wire bus used here to drive the LCD
JWT	JSON Web Token: a signed sign-in token sent with every staff request
LFW	Labeled Faces in the Wild: a public face dataset (13 233 photos of 5 749 people)
ORM	Object-Relational Mapper (Prisma): code that talks to the database
POPIA	Protection of Personal Information Act 4 of 2013
REST	Representational State Transfer: the style of the backend web API
SFace / YuNet	Face recognition model / face detection model from the OpenCV Model Zoo

Abstract

Invigilators at Tshwane University of Technology (TUT) still check every exam candidate by comparing a printed register and a student card with the person at the door. The check is slow, interrupts students who are already writing and depends on the invigilator's eyesight, so impersonation can be missed. This project designed, built and tested the Smart Registration System (SRS): a face-recognition door unit for an exam venue, connected to a custom web console where staff enrol students, book exam halls and watch every entry attempt live.

The proposal planned an ESP32-CAM camera board. Hardware problems with that board led to a redesign: the final door unit is a Windows PC with a USB webcam running OpenCV's YuNet face detector and SFace recogniser, paired with an Arduino Mega that uses an HC-SR04 ultrasonic sensor to take the picture only when the student has stood 25–50 cm from the camera for one second, and shows the result on an LCD with a buzzer. A Node.js/Express backend on Render with a PostgreSQL database on Neon looks up the recognised student's seat for that venue's current exam session, logs every decision, signs students out automatically when the session ends and records an e-mail to every student about their bookings, sent through Brevo once an e-mail account is configured. In a benchmark of the door program's own recognition code on the public LFW dataset, the 0.40 match threshold gave 98.6% verification accuracy with a false acceptance rate of 0.03% and a false rejection rate of 2.8%; one recognition took about 16 ms on a 2.1 GHz processor. All 108 automated backend tests pass. The project met its four objectives and delivered more than the single-venue scope of the proposal.

1. Introduction

1.1 Problem statement

During formal examinations an invigilator must confirm that every person entering the venue is a registered candidate. Today this is done by eye: the invigilator compares the photo on the student card with the face in front of them and ticks an attendance register. The process is slow at the door, it is repeated during the exam (disturbing students who are writing), and a convincing impersonator can pass. There is also no reliable time-stamped record of who entered and when.

1.2 Motivation

Exam integrity is central to the value of a qualification. A system that verifies identity automatically at the door reduces the invigilator's workload, removes the mid-exam interruption, makes impersonation harder and produces an audit trail that can be checked afterwards.

1.3 Why the problem is not yet solved and why it is difficult

Commercial face-recognition access systems exist (Section 3), but they are built for offices, estates or events, are sold on subscription and do not know about exam timetables, seats or invigilators. The problem is difficult because recognition must be fast (a queue of students), accurate under uneven venue lighting, must never let a stranger in by elimination, must keep working when Wi-Fi drops, and handles biometric data that is protected special personal information under POPIA [13], [14].

1.4 Approach and assumptions

The SRS places a door unit at each venue entrance. An ultrasonic sensor waits until the student stands still at the right distance; the PC then recognises the face against the faces enrolled on that PC and looks the student up in the backend's session list (seat, module, already inside, session ended). The result is shown on the LCD and sent to the backend, which logs it and shows it on the invigilators' dashboard. The solution assumes one door unit per venue with mains power and internet access, students who consent to enrolment, and that the invigilator keeps the final say: every denied result falls back to the normal manual card check.

1.5 Main results and contributions

- A working door unit (PC + webcam + Arduino distance/LCD/buzzer unit) with an offline queue and a built-in hardware test.
- A custom REST backend and PostgreSQL schema (11 tables) with staff sign-in, two roles, device keys, exam bookings, seat allocation, automatic sign-out at 12:00/16:00 and student e-mails.
- A web console (SRS Console) with a live dashboard, enrolment, students, modules, bookings, calendar, e-mail log and accounts.
- Measured recognition performance: 98.6% accuracy, FAR 0.03%, FRR 2.8% (1:1, LFW); 97.5% correct entries in a 100-student door simulation.
- 108 of 108 automated backend tests passing.

1.6 Organisation of this report

Section 2 gives the background on face recognition and the hardware used. Section 3 compares the SRS with existing products. Section 4 covers project management and the change from the ESP32-CAM. Section 5 is the technical design of the hardware, door program, backend and web console. Section 6 presents the test results and analysis, Section 7 the challenges, Section 8 the budget, Section 9 the deliverables, Section 10 recommendations and Section 11 the conclusions. Appendices contain the wiring, the device API and the test procedure.

2. Background

2.1 Face detection and face recognition

Face recognition is done in two steps. **Detection** finds where a face is in the picture; **recognition** decides whose face it is. The SRS uses YuNet, a very small convolutional neural network that finds faces and five landmarks (eyes, nose tip, mouth corners) in milliseconds [8]. The landmarks are used to rotate and crop the face to a standard 112×112 pixel image. SFace [9] then turns that image into an **embedding**: 128 numbers that describe the face. Both models are published in the OpenCV Model Zoo [10] and run through OpenCV's built-in `FaceDetectorYN` and `FaceRecognizerSF` classes.

Two embeddings are compared with **cosine similarity**, the cosine of the angle between the two vectors. For normalised embeddings a and b it is simply the dot product:

$$\text{similarity} = a \cdot b = \Sigma\, a_i b_i \quad (i = 1 \ldots 128), \quad \text{with } |a| = |b| = 1$$

A value close to 1 means the same person; a value near 0 means unrelated faces. A **threshold** turns the value into a decision. SFace's published threshold is 0.363; the SRS

uses a slightly stricter 0.40, because letting a stranger into an exam is worse than asking an enrolled student for their card.

2.2 Measuring a biometric system

Biometric systems are judged by two error rates [12]. The **False Acceptance Rate (FAR)** is the share of impostor attempts that are wrongly accepted. The **False Rejection Rate (FRR)** is the share of genuine attempts that are wrongly refused. Raising the threshold lowers FAR and raises FRR. The **Equal Error Rate (EER)** is the point where both are equal and is a common single-number summary.

2.3 Ultrasonic distance gating

The HC-SR04 module sends a 40 kHz sound pulse and measures the time t until the echo returns [16]. Sound travels at about 343 m/s at 20 °C, and the pulse travels to the person and back, so the distance is:

$$d = (343\ \text{m/s} \times t) / 2 \quad \Rightarrow \quad d\ [\text{cm}] \approx t\ [\mu\text{s}] / 58$$

For the SRS window of 25–50 cm the echo time is therefore about 1.45–2.9 ms. The sensor needs roughly 60 ms between pings, so the Arduino measures about 16 times per second and reports the distance to the PC four times per second. Taking the picture only inside this window gives a face that is large and sharp enough for recognition and stops the camera from scanning people who are just walking past.

2.4 Web technologies

The backend is a **REST API**: the web console and the door unit send HTTPS requests with JSON data to fixed addresses such as `/api/access-events`. Staff prove who they are with a **JSON Web Token (JWT)** [25] that the server signs at sign-in; passwords are stored only as **scrypt** hashes [26], a deliberately slow function that makes stolen hashes hard to crack. Door units prove who they are with a secret device key in the `X-Device-Key` header.

3. Past/Related work

Face-recognition access control is already sold commercially. Facephi offers identity verification for public administrations [1]. Page Automation's SmartVisitor is a South African cloud visitor-management system that links doors and turnstiles to face recognition for about R480 per month [2], [3]. Navon Innovations offers face-registered event access with encrypted templates [4]. Openitem checks residents of estates and parks against their profile photo [5]. Table 2 compares these products with the SRS.

Table 2: Comparison of the SRS with existing face-recognition access products

Dimension	Commercial products [1]–[5]	Smart Registration System
Target setting	Offices, estates, events, visitors	University exam venues
Knows exam timetable and seats	No	Yes – shows each student's seat and module; flags "No seat" for the invigilator
Once-per-session rule and sign-out	No	Yes – "already inside", automatic sign-out at 12:00 / 16:00
Where face templates are kept	Usually central cloud	Only on the door PC (data minimisation, POPIA)
Works offline	Varies	Yes – decisions continue; events queued and uploaded later without duplicates
Cost	Subscription (about R480/month for one product)	About R1 225 of hardware on an existing PC; free hosting tiers
Manual fallback recorded	Not exam-specific	Override button records who let a denied

Dimension	Commercial products [1]–[5]	Smart Registration System
		student in and why

The novelty of the SRS is therefore not the face recognition itself but its integration with exam administration: recognition is only the first step, and the result is tied to the exam booking, the seat list and the session time (seat number, once-per-session entry and automatic sign-out). A face is never matched "by elimination": the door program first compares it with every face on the PC and only then looks at the session list.

4. Project Management

4.1 Strategy and task breakdown

The project was done by one student, so it used an Agile "Scrum-lite" approach (as planned in the design document): short sprints, each ending with a working part, with all code in one Git repository and most changes delivered as pull requests. The work was broken down into the tasks in Table 3.

Table 3: Work breakdown and status

#	Task	Outcome
1	Problem definition, literature review, proposal	Approved proposal (20 Aug 2026)
2	Hardware design and design document	Design document submitted (Sep 2026)
3	ESP32-CAM firmware prototype	Stopped because of hardware problems; replaced by task 6
4	Backend REST API and database (Node.js, Prisma, PostgreSQL)	Done – 11 tables, 108 automated tests
5	Web console (React)	Done – 14 screens, two roles
6	PC door program + Arduino door unit	Done – recognition, offline queue, hardware test
7	Exam bookings, seats, e-mails, automatic sign-out	Done (beyond the original scope)
8	Testing, benchmark, final report, presentation	Done – this report

4.2 Timeline

Figure 1 compares the proposal's timetable with what actually happened (taken from the Git history: 104 commits between 18 September and 31 October 2026). The backend and web console started on schedule. The ESP32-CAM work ran into hardware problems and was replaced in the second half of October by the PC and Arduino door unit, which pushed the integration work into the last two weeks before the demonstration.

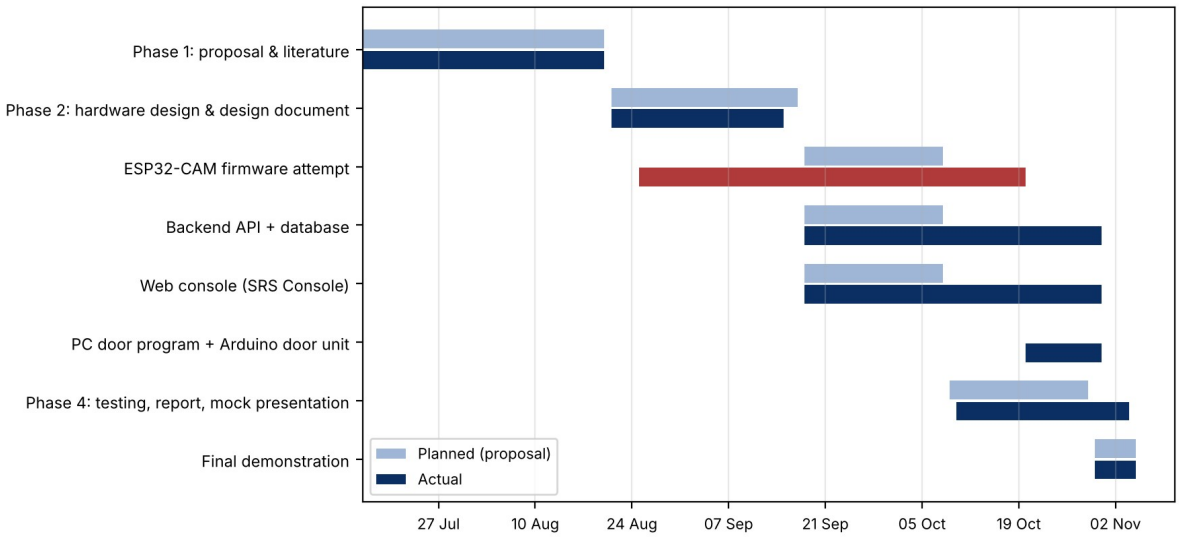


Figure 1: Planned (proposal) versus actual project timeline

4.3 Changes from the original plan

Table 4: Changes from the proposal and design document

Area	Proposal / design	Final system	Reason
Door hardware	ESP32-CAM (OV2640) on batteries	Windows PC + USB webcam + Arduino Mega door unit	Hardware problems with the ESP32-CAM: the firmware could not be uploaded to the board
Recognition	ESP-WHO on the ESP32	OpenCV YuNet + SFace on the PC	Runs on the new hardware; well documented, fast
Result display	LED and buzzer	16×2 LCD + buzzer; seat number on the PC window	Students can read the result
Dashboard	Blynk / Ubidots, then custom	Custom React web console	Lecturer feedback: no ready-made platforms
Communication	MQTT, then HTTP	HTTPS REST, outbound only from the door	Works behind any router; simpler
Scope	One venue, one device	Many venues and devices, bookings, seats, e-mails, roles	Needed to show each student's seat and session

5. Technical Sections

5.1 System architecture

The SRS has four parts (Figure 2): the door unit at each venue, the backend REST API, the PostgreSQL database and the SRS Console web app. The door unit only makes outbound HTTPS calls, so it works behind any router or firewall; the web console never talks to the door unit directly. The backend is the only part that reads and writes the database.

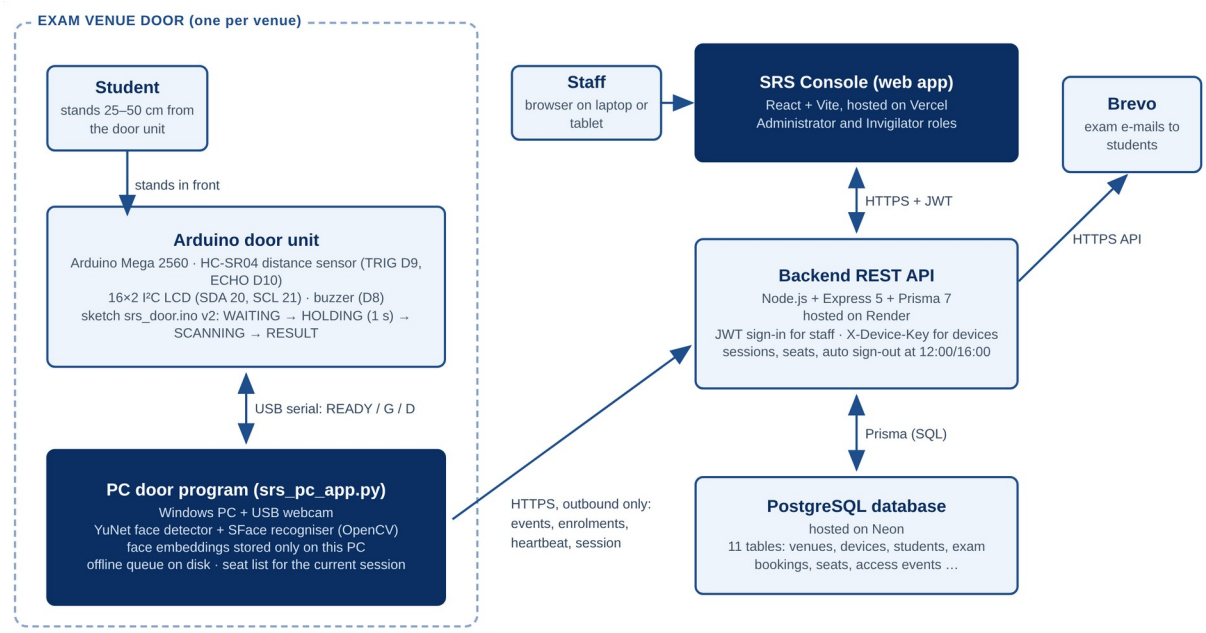


Figure 2: System architecture of the final Smart Registration System

The UML diagrams in this section were drawn as text: the sequence, class and state machine diagrams in Mermaid, and the use case, deployment and activity diagrams in PlantUML, because Mermaid has no UML form for those three.

Figure 3 is the UML use case diagram: the actors (stick figures) and what each can do with the system (ellipses). The Administrator inherits every use case of the Invigilator (hollow-triangle arrow); «include» marks a step that always happens as part of another use case and «extend» one that happens only sometimes.

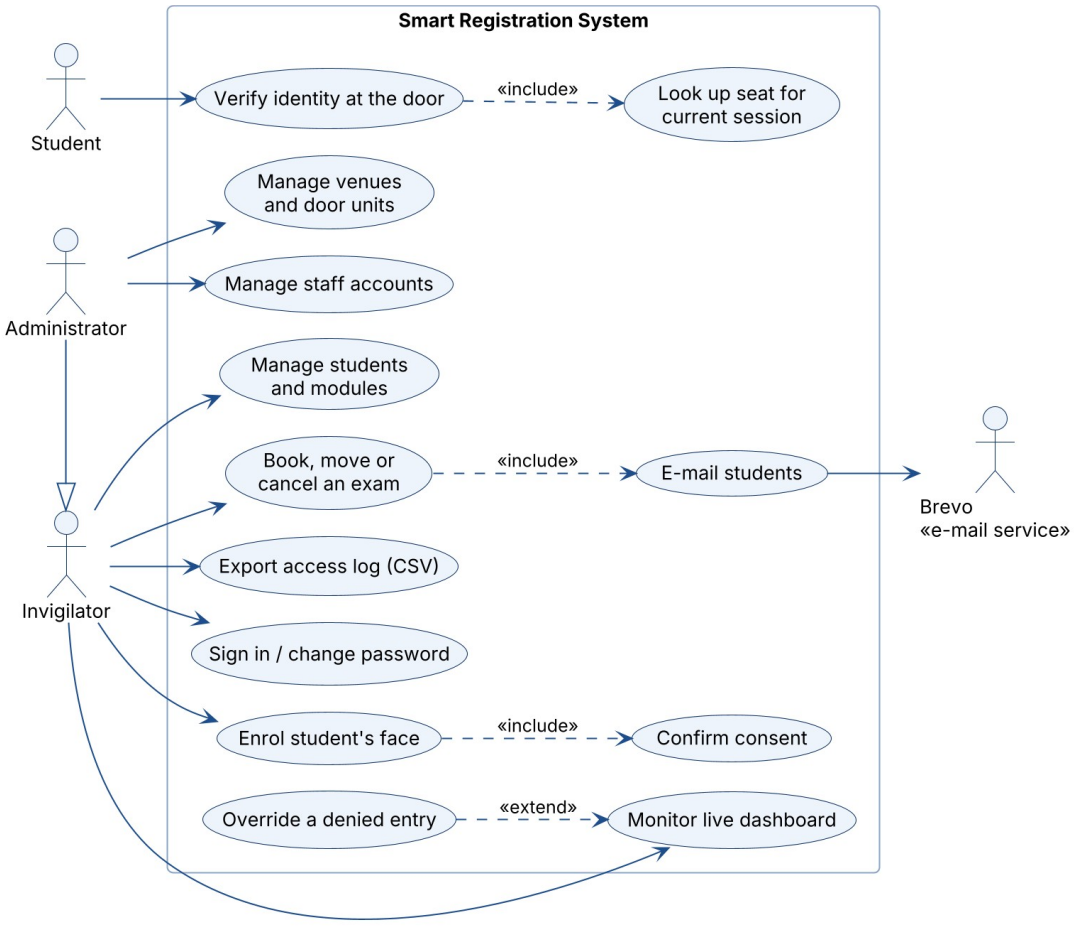


Figure 3: UML use case diagram

Figure 4 is the UML deployment diagram: the 3-D boxes are the machines (nodes), the documents inside them are the software that runs there (artifacts), and the lines are the communication paths with their protocols.

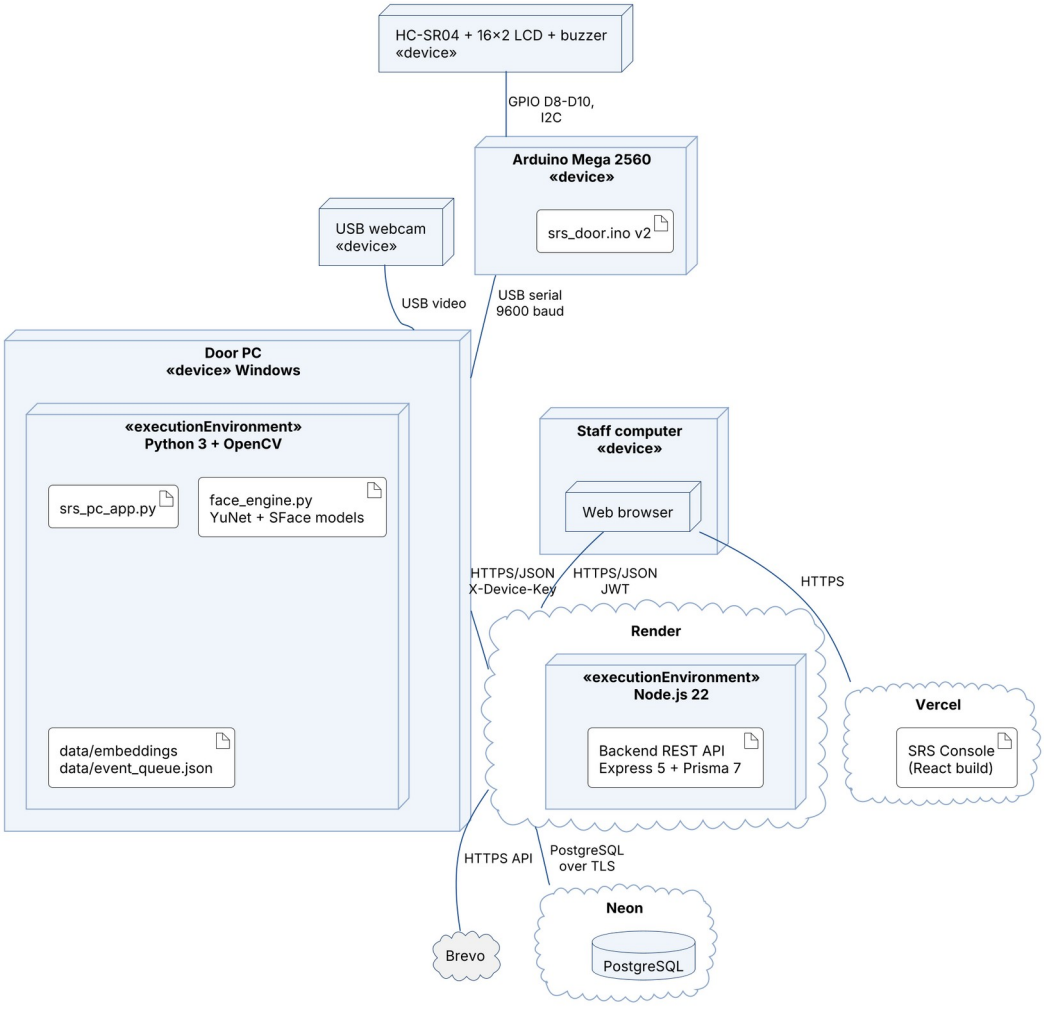


Figure 4: UML deployment diagram

5.2 Hardware design: the door unit

The door unit combines a PC for recognition with an Arduino for the real-time parts (distance, display, sound). The connections are shown in Figure 5 and Table 5.

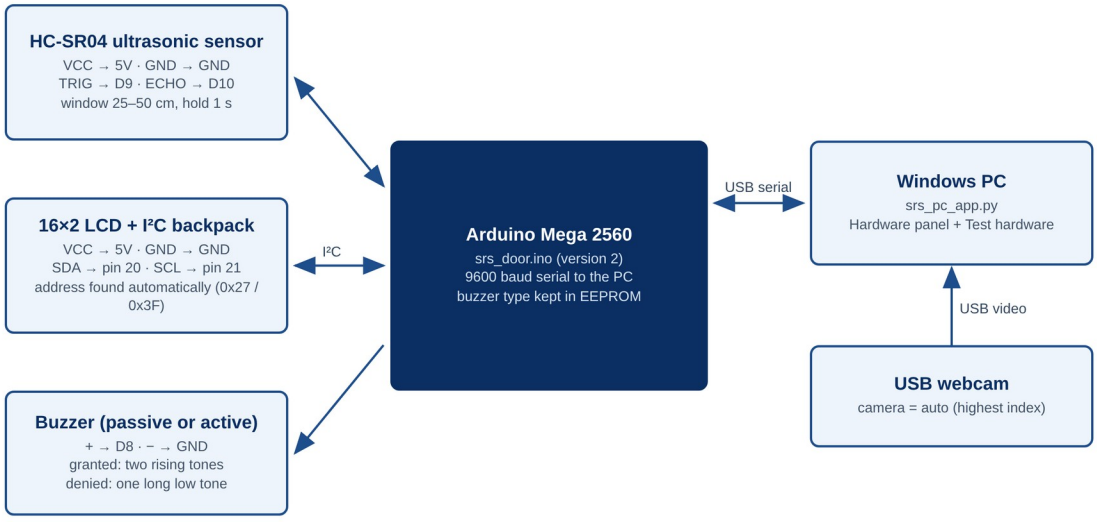


Figure 5: Door unit block and wiring diagram

Table 5: Door unit wiring (Arduino Mega 2560)

Part	Pin on part	Arduino Mega pin	Notes
HC-SR04	VCC / GND	5V / GND	Powered from the Arduino
HC-SR04	TRIG / ECHO	D9 / D10	5 V logic: no divider needed (the ESP32 needed one)
16×2 LCD + I²C backpack	VCC / GND	5V / GND	Contrast set with the screw on the backpack
16×2 LCD + I²C backpack	SDA / SCL	Pin 20 / pin 21	Address found automatically (0x27 or 0x3F)
Buzzer	+ / −	D8 / GND	Passive or active; type stored in EEPROM
PC	USB-B	USB	Power and 9600-baud serial link

5.2.1 Door unit behaviour (srs_door.ino)

The Arduino sketch is a state machine with six states: WAITING (nobody in range), HOLDING (in the 25–50 cm window, counting 1 s; one stray echo shorter than 200 ms does not restart the count), SCANNING (READY sent, waiting up to 6 s for the PC), RESULT (shown for 3 s), LEAVING (the student must step beyond 65 cm before the next scan) and TESTING. The PC answers with one letter: G (granted: two rising tones), D (denied: one long low tone) or A (already inside: two short beeps). Figure 6 is the UML state machine of the sketch: rounded boxes are states with their entry actions (the exact LCD text), and each arrow is labelled event / action. Since version 3 of the sketch the PC app also sends a heartbeat (H) every second: when nothing has come from the PC for 3 s, or the app says goodbye (X) as it closes, the LCD shows NOT CONNECTED / Start the PC app and no scan starts.

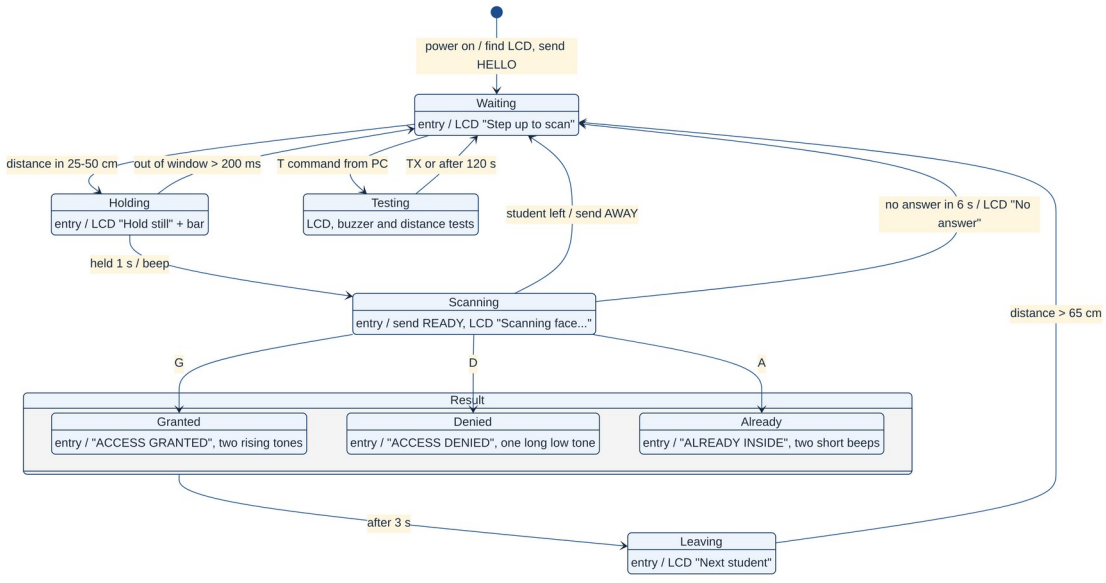


Figure 6: UML state machine of the Arduino door unit (srs_door.ino)

5.2.2 Design calculations

- Echo time for the window: 25 cm × 58 µs/cm = 1.45 ms; 50 cm × 58 µs/cm = 2.9 ms. The sensor's 60 ms ping interval gives about 16 readings per second, so the 1 s hold contains about 16 consistent readings.
- Face size: at 25–50 cm a face fills roughly 150–300 px of a 640×480 webcam image, well above the 90 px minimum the program needs (min_face_px) for a reliable SFace embedding.
- Power: the Arduino, sensor, LCD and buzzer draw well under 200 mA together, inside the 500 mA a USB 2.0 port supplies, so no separate supply or batteries are needed (a key simplification compared with the ESP32-CAM design).

5.3 Door software: the PC door program

`srs_pc_app.py` (Python, OpenCV) turns a Windows PC with a webcam into a door device. It runs four threads: the camera thread (frames, detection, decisions), the backend thread (heartbeat, enrolments, session list, event upload), the Arduino thread (serial messages and the hardware test) and the window (camera view, result banner, seat list, hardware panel and activity log).

5.3.1 Access verification

Figure 7 (UML activity diagram) and Figure 8 (UML sequence diagram) show the decision flow. The face is compared with **every** face on the PC before the session list is consulted, so a stranger cannot be accepted as the "closest" seated student, and the same answer is needed in three frames in a row. An enrolled student is let in once per session: the PC window shows "RECOGNISED" with the seat number and module, or "no exam seat in this session", which the dashboard's Recognised now panel flags as "No seat" for the invigilator. A second scan in the same session gives "ALREADY INSIDE", and a scan after the session has ended gives "SESSION ENDED". Only unknown faces, and faces that match two enrolled students, are denied.

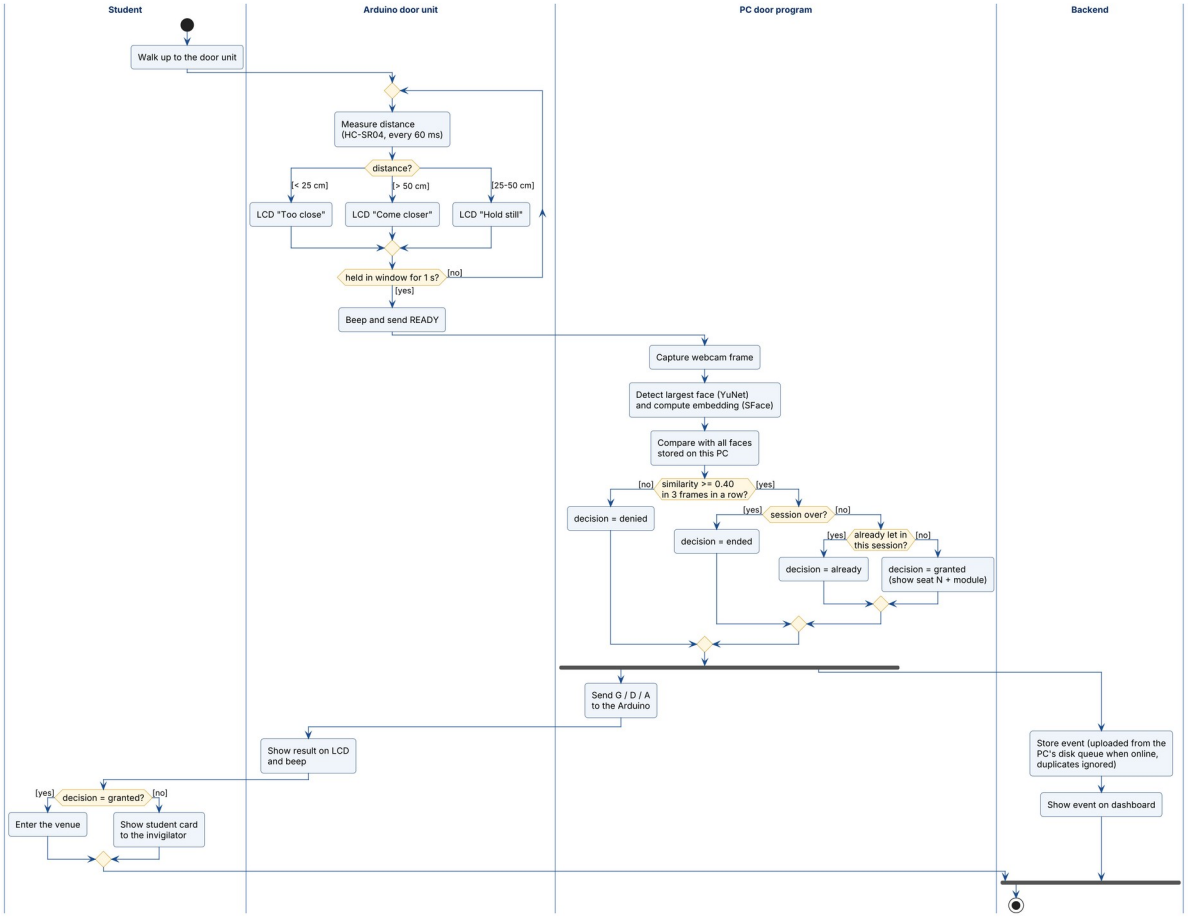


Figure 7: UML activity diagram: access verification, in swimlanes per part

In Figure 7 each swimlane is one part of the system. The black dot is the start and the ringed dot the end; diamonds are decisions with their conditions (guards) in square brackets; the thick bars split the flow into two parallel paths (fork) and join them again. Figure 8 shows the same scan as messages over time: each vertical dashed line is a lifeline, solid arrows are requests, dashed arrows are replies, and the alt frame lists the alternative outcomes.

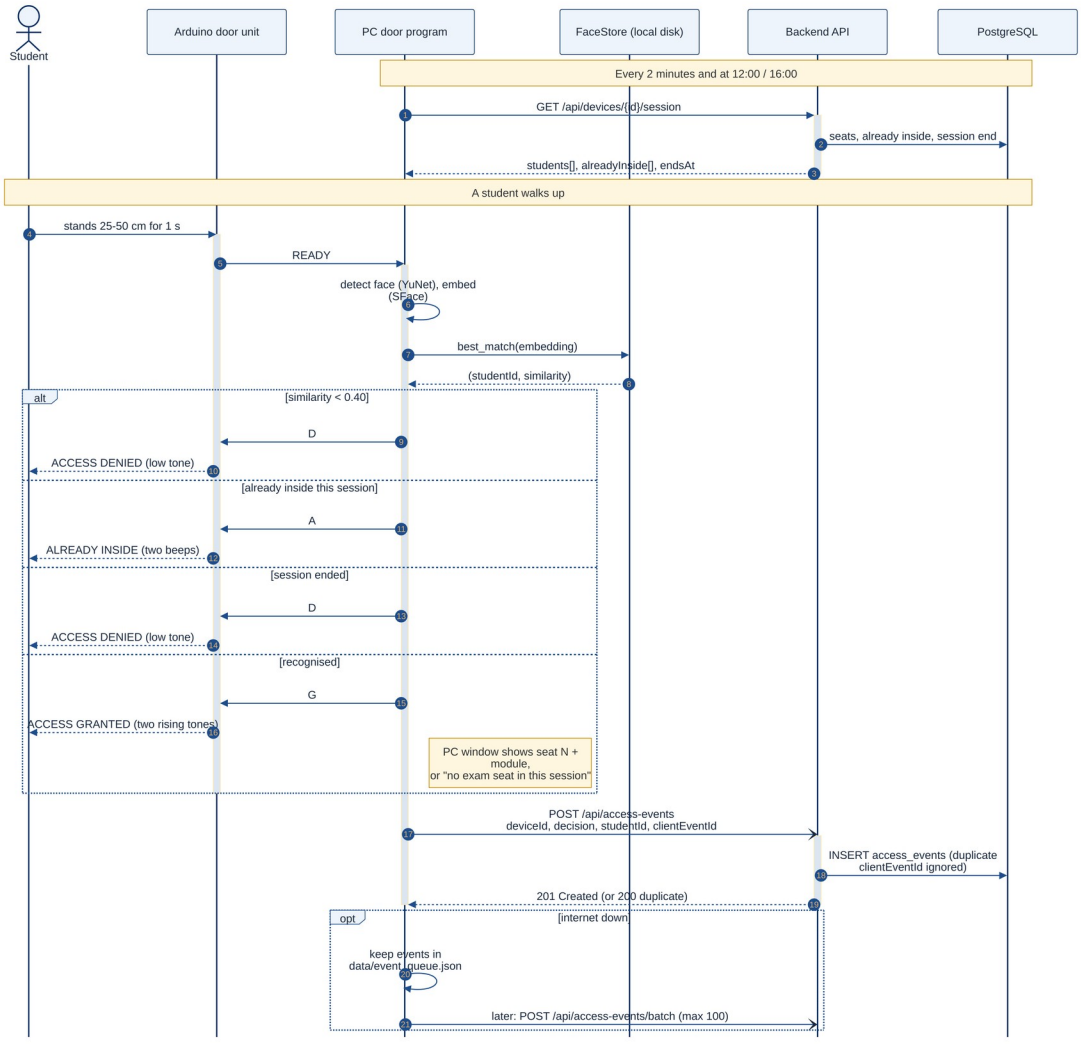


Figure 8: UML sequence diagram: one access verification

5.3.2 Enrolment

Enrolment is started in the web console and carried out at the door (Figure 9, UML sequence diagram; par = parallel, loop = repeated, alt = alternatives). The PC polls the backend every 3 seconds for pending enrolments, captures five shots of a single face that is large enough, averages the five embeddings and stores the result only on the PC as `data/embeddings/<student number>.json`. The backend learns only "enrolled" or "failed" plus one photo that the console shows so staff can check who was enrolled. Before saving, the new face is compared with every face already stored on the PC; if it matches another student (similarity of 0.40 or more), the enrolment is rejected, so one face can only ever belong to one student, and a window shows both faces side by side so the operator can see why.

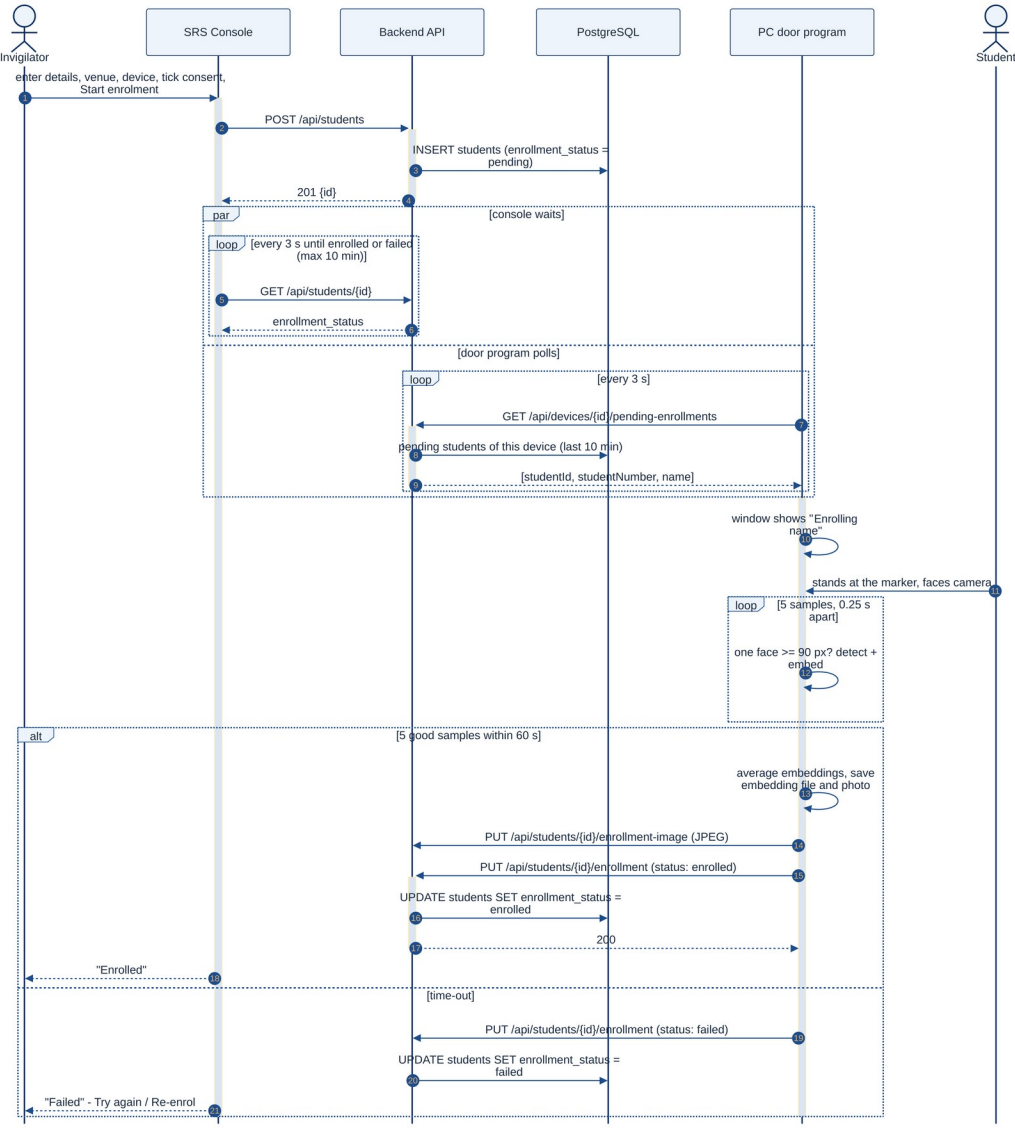


Figure 9: UML sequence diagram: student enrolment

5.3.3 Working offline

Every decision gets a unique `clientEventId`. When the internet is down, events are queued on disk and uploaded later in batches of up to 100 (`POST /api/access-events/batch`). The database has a unique index on (device_id, client_event_id), so an event that is uploaded twice after a Wi-Fi drop is stored only once.

5.4 Backend REST API and database

The backend is written in Node.js with Express 5 and the Prisma 7 ORM and is hosted on Render; the database is PostgreSQL on Neon. It exposes 63 routes in eight groups (Table 6). Database changes are applied automatically at start-up through Prisma migrations.

Table 6: Backend API route groups

Group	Examples	Who may call it
Auth	POST /api/auth/login, PUT /api/auth/password	Everyone (sign-in), signed-in staff
Venues, devices	POST /api/venues, GET /api/devices/:id	Read: staff · change: administrators only
Device API	heartbeat, config, pending-enrollments, roster,	Door units with a valid X-Device-

Group	Examples	Who may call it
	session	Key
Access events	POST /api/access-events (+ /batch), GET, /:id/override	Devices post; staff read and override
Students, modules	POST /api/students, /:id/re-enroll, /:id/modules	Signed-in staff
Bookings	GET /availability, POST /api/bookings	Signed-in staff
E-mails, stats	GET /api/emails, POST /retry, GET /api/stats	Signed-in staff
Users	POST /api/users, /:id/reset-password	Administrators only

Figure 10 is the UML class diagram of the database: one class per table with its columns and types, «PK»/«FK» marking keys, multiplicities (1, 0..1, 0..*) on each end of an association, and filled diamonds for composition (the part is deleted with its owner). Venues have a seat capacity; devices belong to a venue; students are enrolled on a device. An exam booking links a module to a venue, a date and one of two fixed slots (09:00 or 13:00); every student registered for the module gets a seat reservation with a seat number that is unique in that venue and slot. Access events record every decision, and event overrides record manual decisions by invigilators.

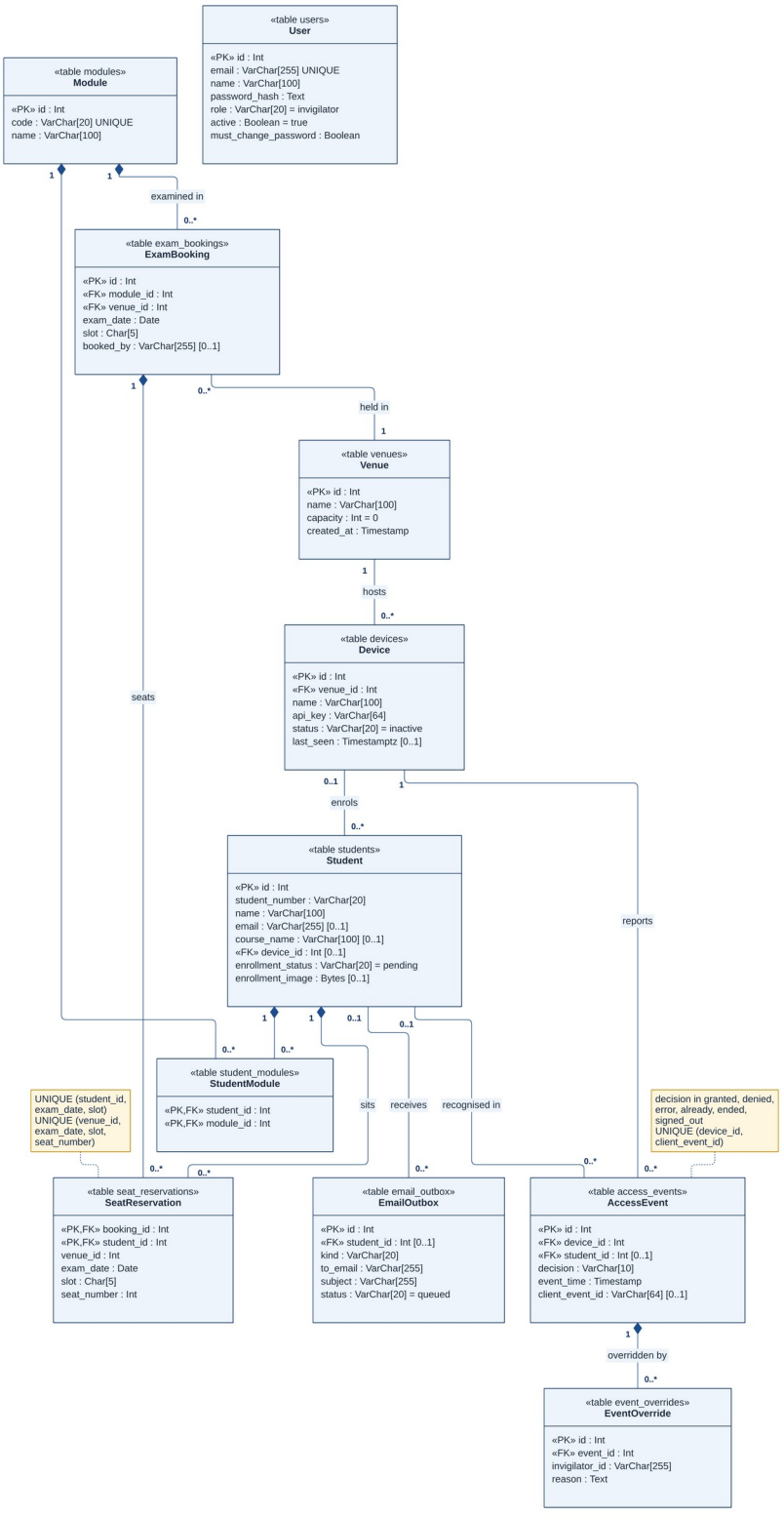


Figure 10: UML class diagram of the database (11 tables)

5.4.1 Business rules

- **Session:** before 12:00 South African time the 09:00 session applies; from 12:00 the 13:00 session.

- **Who is let in:** a student enrolled on this door unit, once per session; the console shows whether they have a seat in the exam running now.
- **Once per session:** a second granted event in the same session is stored as "already".
- **Automatic sign-out:** at 12:00 and 16:00 everyone let in gets a "signed_out" event; later attempts are "ended".
- **Capacity:** several modules may share a venue and slot only while the total seats fit the venue capacity; a student cannot have two exams in the same slot.
- **E-mails:** booking, moving or cancelling an exam writes an e-mail to the outbox, which is sent through Brevo [24] in the background, so bookings never wait for e-mail.

Figure 11 shows how the two sessions of an exam day and the automatic sign-out fit together, and Figure 12 is the UML sequence diagram of booking an exam, e-mailing the students and the sign-out timer.

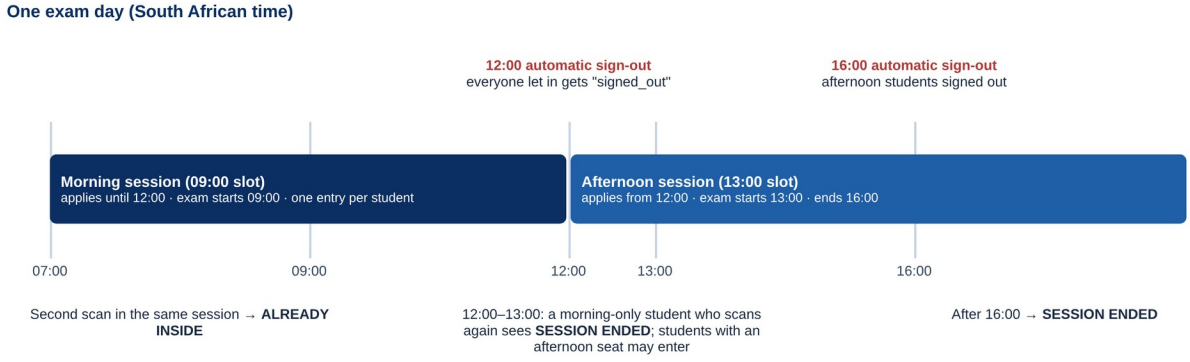


Figure 11: Exam sessions, once-per-session entry and automatic sign-out

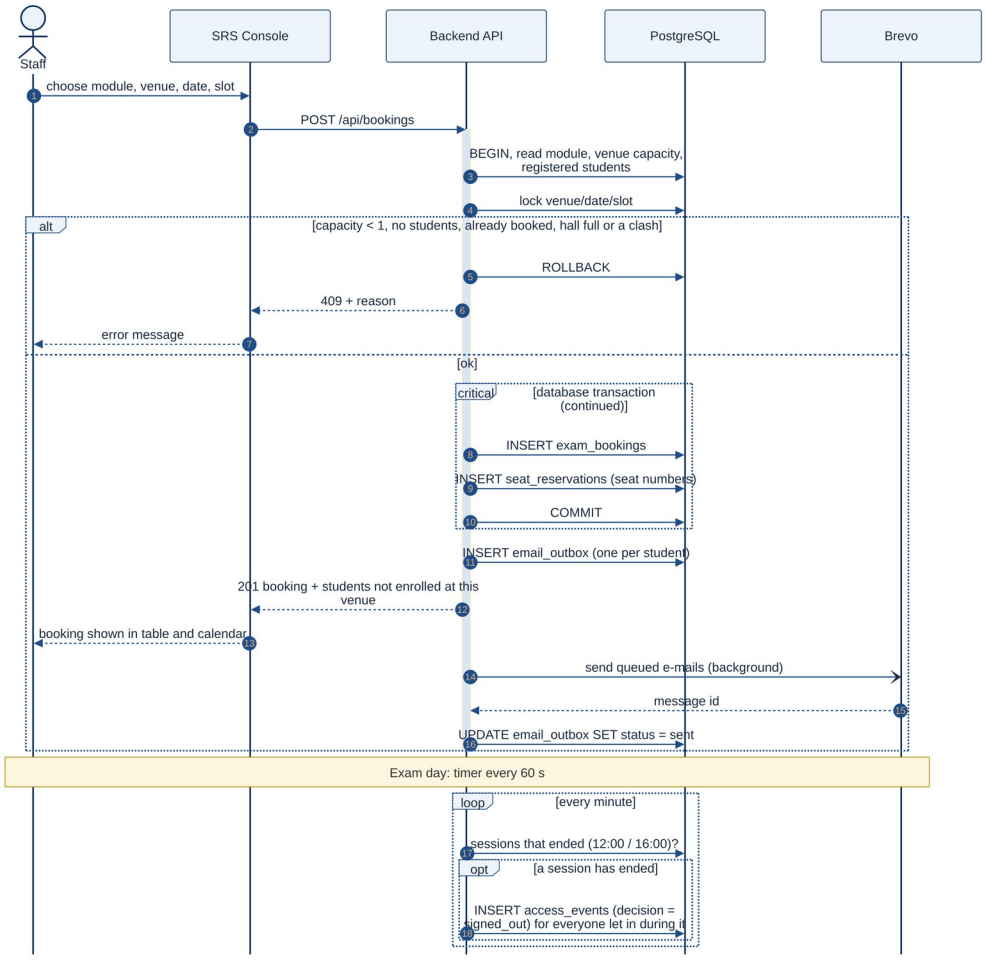


Figure 12: UML sequence diagram: booking an exam, e-mails and automatic sign-out

5.4.2 Security and privacy

- Staff sign in with e-mail and password; passwords are hashed with scrypt (N = 16384, r = 8, p = 1).
- Sign-in tokens (JWT, HS256) expire after 12 hours; after 5 failed sign-ins an account is blocked for 15 minutes.
- New accounts and reset passwords get a temporary password that must be changed at first sign-in.
- Two roles: administrators manage venues, devices, keys and accounts; invigilators cannot see device keys.
- An administrator can replace a device key at any time; the old key stops working at once.
- Face embeddings never leave the door PC; the database stores only one enrolment photo per student, shown on the console so staff can check who was enrolled. Consent is confirmed with a tick box before every enrolment (POPIA); the tick is not yet stored as a record (see Recommendations).

5.5 Web console (SRS Console)

The console is a React 19 single-page app built with Vite and hosted on Vercel. Figures 13 to 18 show the main screens (demonstration data only).

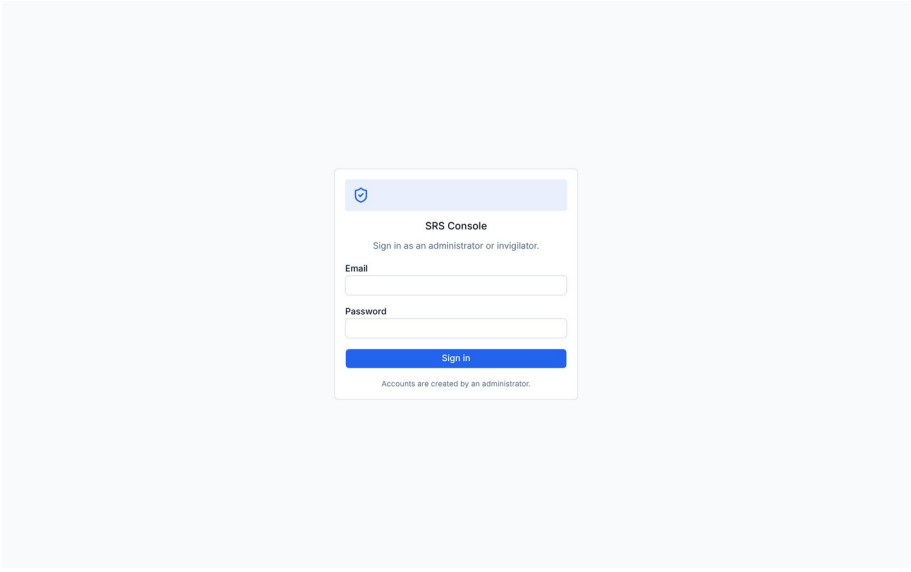


Figure 13: Sign-in screen

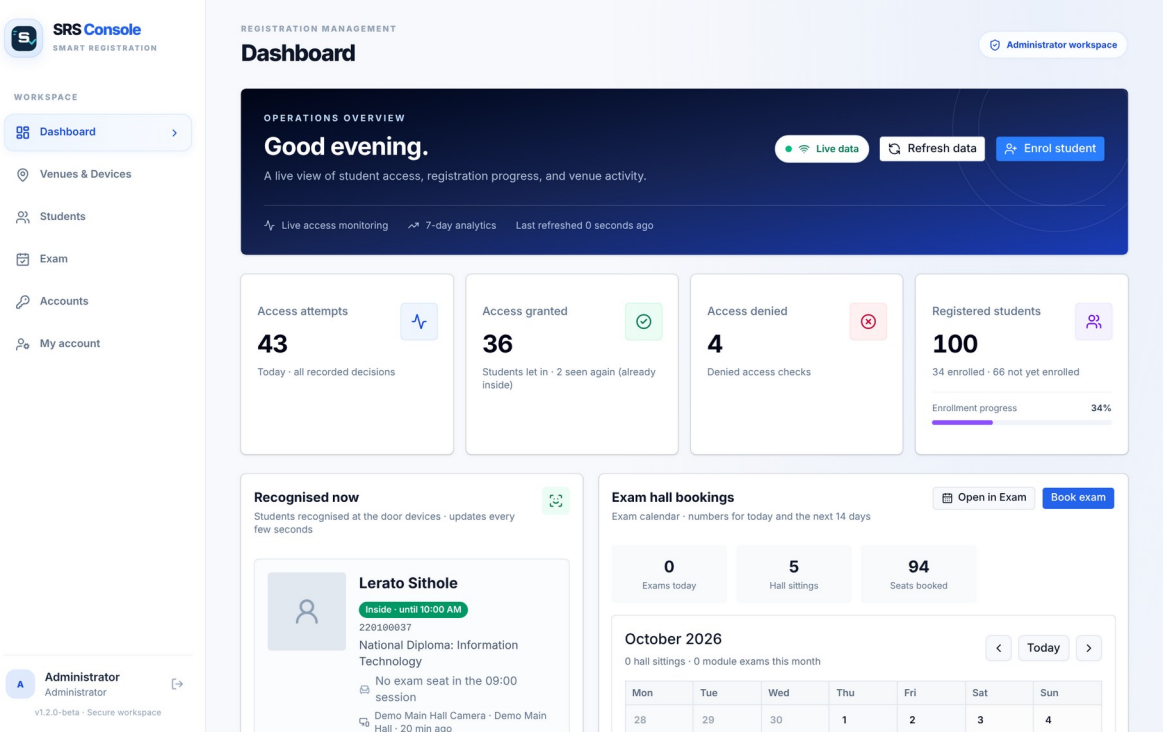


Figure 14: Dashboard: live statistics, students recognised now and exam hall bookings

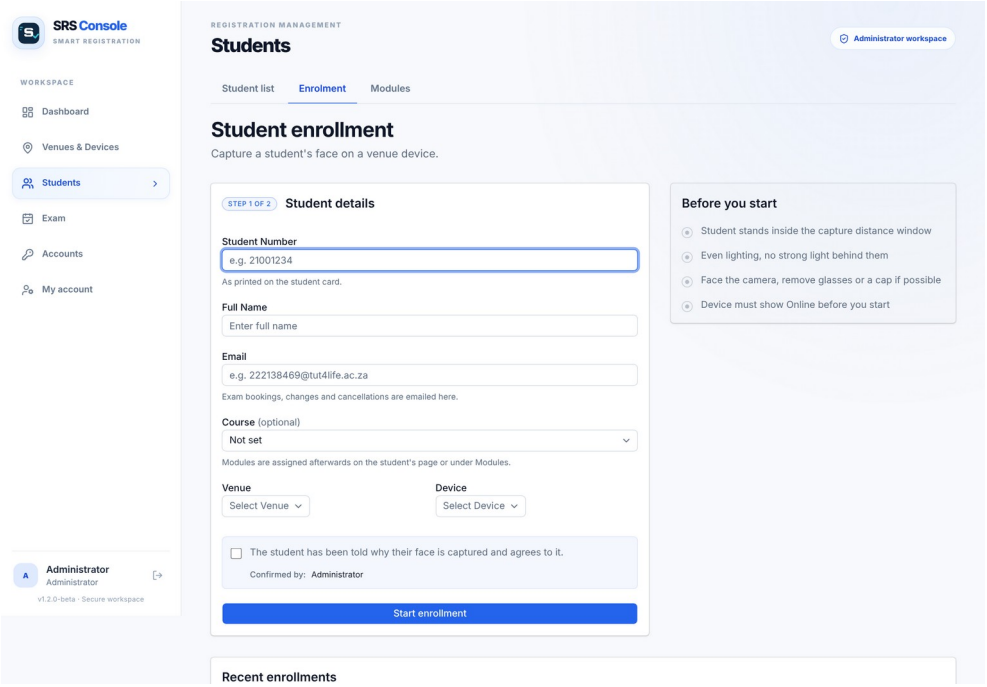


Figure 15: Student enrolment with consent confirmation

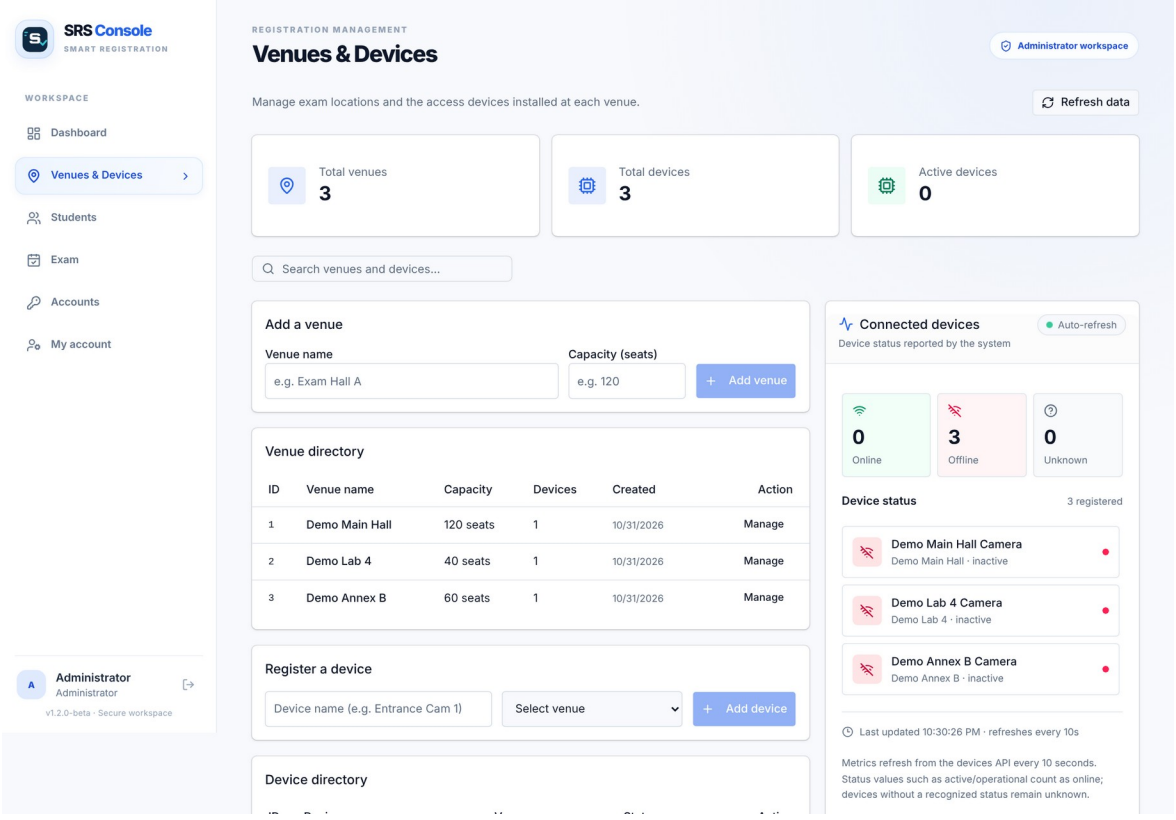


Figure 16: Venues and devices

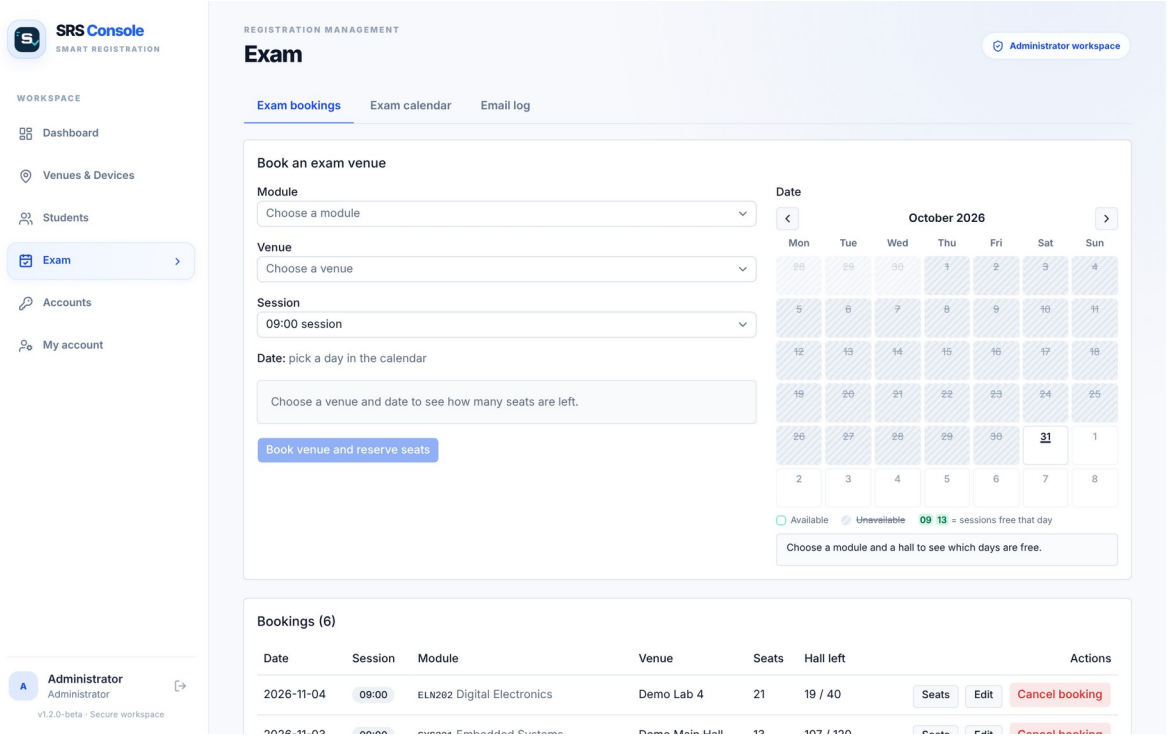


Figure 17: Exam hall bookings

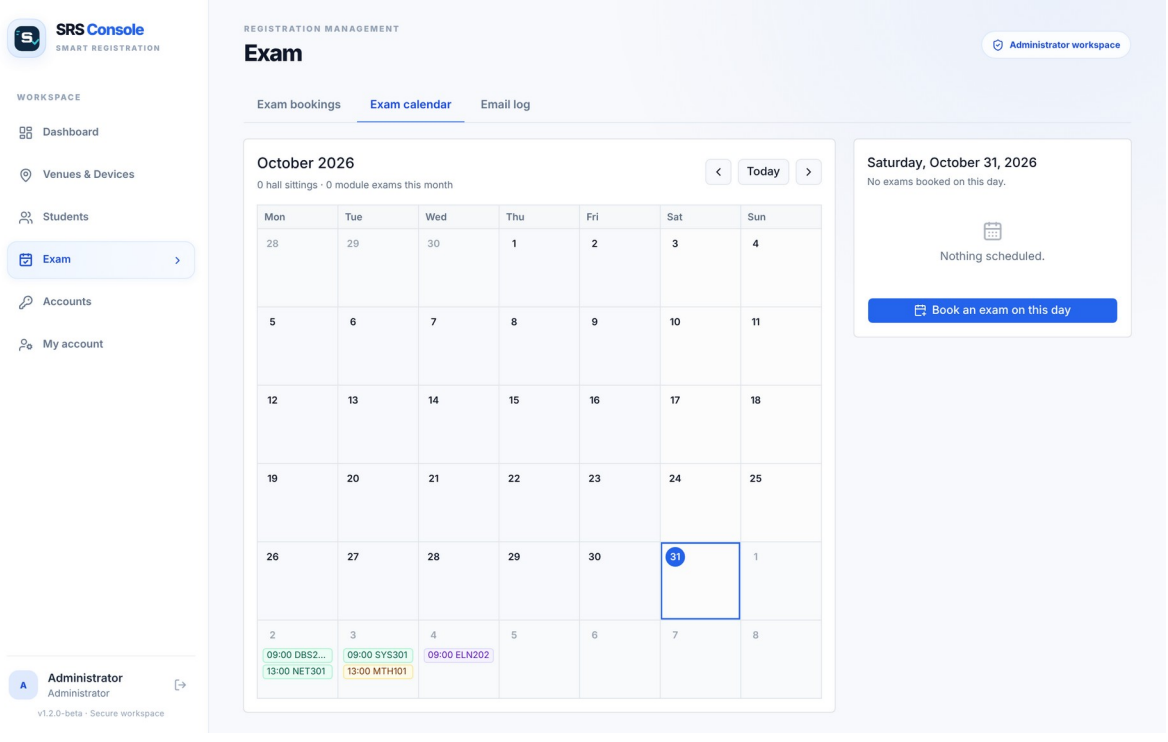


Figure 18: Exam calendar

6. Project Results and Analysis

6.1 Test strategy

Testing was done at three levels: automated tests of the backend, a benchmark of the recognition code on a public dataset, and functional tests of the complete system with the

door unit. Table 7 lists what was tested. A GitHub Actions workflow runs all backend tests against a fresh PostgreSQL database, plus the web console's lint and build, on every change.

Table 7: Automated backend tests (node:test) – run on 1 November 2026

Test file	What it checks	Tests	Result
api.test.js	Venues, devices, students, enrolment flow, device API, access events, batch upload	38	38 passed
booking.test.js	Capacity, modules, seat allocation, slot clashes, availability, exam e-mails	26	26 passed
extra.test.js	Edits, validation edge cases, statistics arithmetic, limits, device key replacement, device moves	26	26 passed
signout.test.js	Session end at 12:00 / 16:00, sign-out events, "session ended"	5	5 passed
auth.test.js	Sign-in, roles, lock-out, password change, hidden device keys	13	13 passed
Total		**108**	**108 passed, 0 failed**

ESLint (a code-style and bug checker) also reports no problems in the backend or the web console.

6.2 Recognition benchmark (FAR and FRR)

To measure recognition accuracy objectively, the door program's own `face_engine.py` (YuNet detector with score threshold 0.85, SFace recogniser, largest face first, threshold 0.40) was run on all 13 233 photos of 5 749 people in the public Labeled Faces in the Wild dataset [11]. Only one photo had no detectable face. Two tests were run:

- **1:1 verification:** 3 000 pairs of photos of the same person and 3 000 pairs of different people (random, seed 2026).
- **1:N door simulation:** 100 randomly chosen "students" enrolled with one photo each; their other photos are genuine attempts (about 386 per run) and 1 000 photos of people who are not enrolled are impostor attempts; repeated 5 times.

Table 8: 1:1 verification results for different thresholds (LFW, 6 000 pairs)

Threshold	FAR (%)	FRR (%)	Accuracy (%)	Note
0.3	0.70	1.37	98.97	
0.34	0.30	1.67	99.02	
0.363	0.10	2.13	98.88	SFace published value
0.4	0.03	2.83	98.57	**SRS setting**
0.45	0.00	5.13	97.43	
0.5	0.00	9.07	95.47	

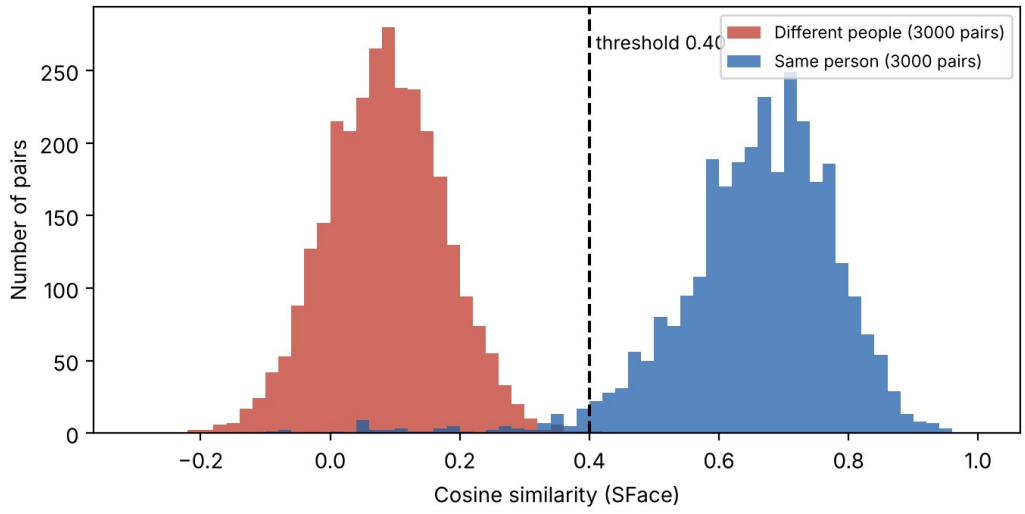


Figure 19: Similarity scores of same-person and different-person pairs

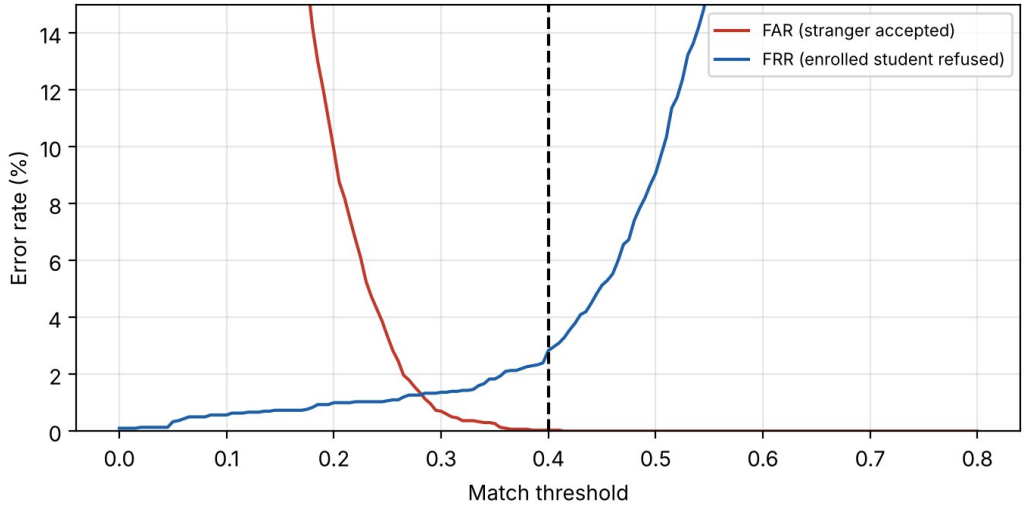


Figure 20: FAR and FRR against the match threshold

Table 9: 1:N door simulation (100 enrolled students, mean ± standard deviation over 5 runs)

Measure	Result
Genuine attempts let in as the right student	97.5% ± 1.3
Genuine attempts refused (FRR)	2.2% ± 1.0
Genuine attempts matched to the wrong student	0.30% ± 0.37
Strangers matched to some enrolled student (FAR, before the seat check)	1.8% ± 0.5

6.2.1 Analysis

The two score distributions in Figure 19 hardly overlap: same-person pairs average 0.66 and different-person pairs 0.08. The equal error rate is 1.25% at a threshold of 0.28. Moving from the published 0.363 to the SRS setting of 0.40 cuts FAR from 0.10% to 0.03% (one false accept in 3 000) at the cost of FRR rising from 2.1% to 2.8%. For an exam door this is the right trade-off: a refused student is simply checked by hand, while an accepted impostor defeats the purpose of the system.

In the door simulation FAR rises to about 1.8% because each stranger is compared with

100 faces instead of one. Because the door lets any recognised student in, this is the figure that matters at the door; it is kept down by holding only the students enrolled on that door unit (the roster clean-up) and by the invigilator, who sees the matched name and seat on the dashboard. LFW photos are also harder than the door (news photos with varied pose, lighting and age, and a single enrolment photo instead of five averaged shots taken at the door), so the door is expected to perform at least as well. These figures must still be confirmed on site with the procedure in Appendix C.

6.3 Speed

On a 2.1 GHz Intel Xeon processor (4 cores) YuNet detection took on average 2.4 ms and alignment plus SFace embedding 7.3 ms per LFW photo. On full 640×480 webcam-sized frames the complete pipeline (detect, align, embed, compare with 100 faces) took 15.6 ms on average (95th percentile 19.3 ms, maximum 35 ms, 300 frames). The time a student waits is therefore dominated by the deliberate 1 s hold at the sensor; recognition adds well under 0.1 s, comfortably inside the proposal's 2-second target.

6.4 Achievement of the objectives

Table 10: Proposal objectives and results

Objective (proposal)	Result	Status
Identify and verify a face within 2 s with 90%+ accuracy, gated by ultrasonic distance	≈ 1 s hold + ≈ 16 ms recognition; 98.6% accuracy (LFW)	Met (benchmark)
Keep FAR and FRR acceptable	FAR 0.03%, FRR 2.8% at 0.40	Met (benchmark)
Log time, student and decision for every attempt; live dashboard	Every decision stored; dashboard refreshes every 15 s, "recognised now" every 3 s; CSV export	Met
Manual override when recognition fails	Override button records invigilator and reason	Met

6.5 Functional tests of the complete system

The PC door program has a built-in **Test hardware** window that checks the door unit step by step. Table 11 lists the results.

Table 11: Door unit functional test (Test hardware window and end-to-end checks)

#	Check	Expected	Result
1	Arduino connected over USB	Port shown, sketch version 3	Success
2	Door unit program answers	PONG v3	Success
3	LCD found on the I²C bus	T LCD OK 0x27 / 0x3F	Success
4	LCD shows the test screen	Yes	Success
5	Buzzer beeps	Three beeps	Success
6	Distance sensor gives readings	T SONAR OK, ≥ 8/10 good	Success
7	Sees a hand at 25–50 cm	Distance shown in green	Success
8	Enrol a student from the console	Page changes from "Capturing…" to "Enrolled"	Success
9	Enrolled student with a seat walks up	ACCESS GRANTED on LCD; seat N on the PC window and dashboard	Success
10	Unenrolled person walks up	DENIED; Override available on the dashboard	Success
11	Wi-Fi off, three entries, Wi-Fi on	All three events appear once on the dashboard	Success

7. Challenges Faced

- **ESP32-CAM hardware problems.** The planned ESP32-CAM board could not be made to work reliably. The firmware could not be uploaded to the board. Impact: about three weeks of Phase 3 were lost. Solution: the door unit was redesigned around a PC and webcam for recognition and an Arduino Mega for the sensor, LCD and buzzer, which removed the memory, power and pin limits of the ESP32-CAM.
- **Late integration.** Because the new door unit was only built in the last two weeks of October, testing and documentation had to be compressed. The automated backend tests (108) and a device simulator made it possible to test the console and backend before the hardware was ready.
- **Slow hosting wake-up.** The free Render tier sleeps; the first request can take 30–60 s. The door program uses long time-outs and retries, and the console tells users to wait on first load.
- **Unreliable Wi-Fi.** Events are queued on disk and uploaded in duplicate-proof batches when the connection returns.
- **Time zones.** Servers run in UTC while exams run on South African time (UTC+2); all session rules use Africa/Johannesburg explicitly.
- **Lecturer feedback on software.** Ready-made dashboards (Blynk/Ubidots) were not accepted as the student's own work, so a custom backend and web console were built from scratch.

8. Final Budget

Table 12 lists the expenses. Prices are **estimated South African retail prices (October 2026)** for the parts used, based on typical prices at local electronics suppliers and online retailers; exact receipts were not available. The ESP32-CAM items use the prices from the approved proposal.

Table 12: Final budget (estimated South African prices)

Item	Qty	Unit price	Total
Final door unit			
Arduino Mega 2560 R3 (compatible board)	1	R350.00	R350.00
HC-SR04 ultrasonic distance sensor	1	R40.00	R40.00
16×2 LCD with I²C backpack	1	R150.00	R150.00
Buzzer (passive)	1	R15.00	R15.00
USB 1080p webcam	1	R400.00	R400.00
USB A-to-B cable for the Arduino	1	R40.00	R40.00
Breadboard and jumper wires	1	R80.00	R80.00
Enclosure / mounting stand	1	R150.00	R150.00
Windows PC (student's own laptop)	1	R0.00	R0.00
Software and hosting (OpenCV, Node.js, Render, Neon, Vercel, Brevo free tiers)	–	R0.00	R0.00
Subtotal: final door unit			**R1 225.00**
ESP32-CAM prototype (not used in the final system)			
ESP32-CAM module	1	R200.00	R200.00
FTDI USB-to-serial programmer	1	R31.00	R31.00
Li-ion pack + charging module	1	R250.00	R250.00
Buck converter (step-down to 5 V)	1	R45.00	R45.00
Subtotal: ESP32-CAM prototype			**R526.00**
Total spent			**R1 751.00**
Proposal budget			R1 000.00

Item	Qty	Unit price	Total
Over budget			**R751.00**

Justification. The final door unit alone (R1 225) is close to the R1 000 budget; the overrun comes from buying two sets of hardware after the ESP32-CAM had to be abandoned. The webcam (R400) and Arduino Mega (R350) are the largest items. The proposal's contingency (R250–R313) was used up by the change. Re-using the student's own laptop and free hosting tiers kept the software cost at zero.

9. Project Deliverables

Table 13: Planned and delivered outcomes

Deliverable	Planned	Delivered
Door device with distance-gated face recognition	ESP32-CAM + HC-SR04	PC + webcam + Arduino Mega with HC-SR04, LCD and buzzer
Access log and live dashboard	IoT dashboard for one room	Custom web console for any number of venues and devices
Manual override	Backup method	Override button with recorded reason
Backend and database	Custom Node.js/Express + PostgreSQL	Delivered, 11 tables, 63 routes, 108 automated tests
Exam administration	Not planned	Modules, exam bookings, seats, calendar, automatic sign-out, e-mails
Staff accounts	Not planned	Administrator and invigilator roles with secure sign-in
Documents	Proposal, design document, user manual, final report	All delivered, plus this report's benchmark and a presentation
Source code	Git repository	github.com/TebogoClifford/ Smart_Registration_System

Everything set out in the proposal was delivered, with one change in hardware (Section 4.3). The system also goes well beyond the single-venue scope of the proposal.

10. Recommendations and Future Work

- **Seat-based refusal:** add an option to deny students with no seat in the current session (the session list already provides it).
- **On-site accuracy study:** measure FAR/FRR at a real venue entrance with at least 30 volunteers under different lighting (Appendix C).
- **Liveness detection:** add a check that the face is a live person (blink or depth) so a printed photo cannot be used.
- **Embedded door unit:** move the PC program to a small single-board computer (for example a Raspberry Pi 5) to cut size and cost.
- **Link to the university system:** import students, modules and timetables from the student information system instead of typing them.
- **Push updates:** replace dashboard polling with server-sent events or WebSockets for instant updates.
- **Retention policy: delete enrolment photos and embeddings automatically after the exam period, as POPIA requires, and store each student’s consent with the date and the staff member who recorded it.**
- **Paid hosting: move off free tiers so the backend never sleeps during an exam (a scheduled ping now keeps the free backend awake on weekdays from 06:00 to 17:00).**

11. Conclusions

The project set out to replace the manual identity check at the exam venue door with an automatic, auditable face check. The Smart Registration System does this: a door unit recognises a student whose face matches a face enrolled on that unit, lets them in once per session, shows the result on an LCD and their seat number on the PC, and logs every decision to a live web console where invigilators can override denied results.

All four objectives of the proposal were met. Recognition takes about 16 ms after the 1-second distance hold, and the recognition code reached 98.6% accuracy with a false acceptance rate of 0.03% on the public LFW benchmark. The main setback was the ESP32-CAM, whose hardware problems forced a redesign around a PC, webcam and Arduino; this cost time and pushed the budget to an estimated R1 751, but produced a more capable and easier-to-maintain door unit. The system also grew beyond its original scope with exam bookings, seat allocation, automatic sign-out, e-mails and secure staff accounts.

The significance of the work is that it ties face recognition to exam administration, keeps biometric templates on the door unit in line with POPIA, and keeps the invigilator in control. With an on-site accuracy study and liveness detection, the SRS could be piloted at a TUT exam venue.

12. References

[1] Facephi, "Verification and identity management for public administration." [Online]. Available: https://www.facephi.com. [Accessed: 12-Aug-2026].

[2] Page Automation, "SmartVisitor cloud-based visitor management system." [Online]. Available: https://pageauto.co.za. [Accessed: 12-Aug-2026].

[3] "Page Automation helps clients keep track of visitors with SmartVisitor software," ITWeb, Aug. 2020. [Online]. Available: https://www.itweb.co.za. [Accessed: 13-Aug-2026].

[4] Navon Innovations, "Face recognition event access system." [Online]. Available: https://www.navoninnovations.com. [Accessed: 14-Aug-2026].

[5] Openitem, "Face ID recognition for estate and park access control." [Online]. Available: https://www.openitem.co.za. [Accessed: 15-Aug-2026].

[6] Espressif Systems, "ESP-WHO: face detection and recognition framework." [Online]. Available: https://github.com/espressif/esp-who. [Accessed: 31-Oct-2026].

[7] Espressif Systems, "ESP-IDF Programming Guide." [Online]. Available: https://docs.espressif.com/projects/esp-idf/. [Accessed: 31-Oct-2026].

[8] W. Wu, H. Peng and S. Yu, "YuNet: A tiny millisecond-level face detector," Machine Intelligence Research, vol. 20, no. 5, pp. 656–665, 2023.

[9] Y. Zhong, W. Deng, J. Hu, D. Zhao, X. Li and D. Wen, "SFace: Sigmoid-constrained hypersphere loss for robust face recognition," IEEE Transactions on Image Processing, vol. 30, pp. 2587–2598, 2021.

[10] OpenCV, "OpenCV Zoo: face_detection_yunet and face_recognition_sface."

[Online]. Available: https://github.com/opencv/opencv_zoo. [Accessed: 31-Oct-2026].

[11] G. B. Huang, M. Ramesh, T. Berg and E. Learned-Miller, "Labeled Faces in the Wild: A database for studying face recognition in unconstrained environments," University of Massachusetts, Amherst, Tech. Rep. 07-49, Oct. 2007.

[12] ISO/IEC 19795-1:2021, Information technology – Biometric performance testing and reporting – Part 1: Principles and framework. Geneva: ISO, 2021.

[13] Republic of South Africa, Protection of Personal Information Act 4 of 2013. Pretoria: Government Printer, 2013.

[14] Information Regulator (South Africa), "Guidance note on processing of special personal information," 28 Jun. 2021. [Online]. Available: https://inforegulator.org.za. [Accessed: 31-Oct-2026].

[15] Arduino, "Arduino Mega 2560 Rev3 documentation." [Online]. Available: https://docs.arduino.cc/hardware/mega-2560/. [Accessed: 31-Oct-2026].

[16] ElecFreaks, "Ultrasonic ranging module HC-SR04 datasheet." [Online]. [Accessed: 31-Oct-2026].

[17] Prisma Data, "Prisma ORM documentation." [Online]. Available: https://www.prisma.io/docs. [Accessed: 31-Oct-2026].

[18] OpenJS Foundation, "Express 5 documentation." [Online]. Available: https://expressjs.com. [Accessed: 31-Oct-2026].

[19] Meta Open Source, "React documentation." [Online]. Available: https://react.dev. [Accessed: 31-Oct-2026].

[20] PostgreSQL Global Development Group, "PostgreSQL 16 documentation." [Online]. Available: https://www.postgresql.org/docs/16/. [Accessed: 31-Oct-2026].

[21] Neon Inc., "Neon serverless Postgres documentation." [Online]. Available: https://neon.tech/docs. [Accessed: 31-Oct-2026].

[22] Render, "Render documentation: web services." [Online]. Available: https://render.com/docs. [Accessed: 31-Oct-2026].

[23] Vercel, "Vercel documentation." [Online]. Available: https://vercel.com/docs. [Accessed: 31-Oct-2026].

[24] Brevo, "Transactional e-mail API reference." [Online]. Available: https://developers.brevo.com. [Accessed: 31-Oct-2026].

[25] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.

[26] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," RFC 7914, IETF, Aug. 2016.

[27] C. Du and M. Muwanguzi Ntume, Project Design 376 / Industry Exposure Student Course Guide (PD2316D). Pretoria: Tshwane University of Technology, Jul. 2025.

[28] T. C. Mance, "Smart Registration System for Exam Venue Access Based on Face Recognition: Design documentation," Tshwane University of Technology, Sep.

2026.

Appendices

Appendix A: Door unit serial protocol

Table 14: Serial messages between the Arduino and the PC (9600 baud, one per line)

Direction	Message	Meaning
Arduino → PC	HELLO SRS-DOOR v3 LCD=0x27 BUZ=P	Start-up: version, LCD address, buzzer type
Arduino → PC	S <cm> <state>	Distance and state, four times a second
Arduino → PC	READY / AWAY	Student held still for 1 s: take the picture / student left: cancel
PC → Arduino	G / D / A	Granted / denied / already inside
PC → Arduino	P → PONG v3	Are you there?
PC → Arduino	H / X	Heartbeat every second / the PC app is closing; after 3 s without a message the LCD shows NOT CONNECTED
PC → Arduino	TL, TB, TS, TX	Hardware test: LCD, buzzer, sonar, end of test

Appendix B: Device API used by the door program

Table 15: Device API endpoints (header X-Device-Key on every call)

Method and path	Purpose
GET /health	Wait until the backend is awake
POST /api/devices/heartbeat	Mark the device online; returns server time
GET /api/devices/{id}/config	Settings: venue, poll interval, batch size
GET /api/devices/{id}/pending-enrollments	Students waiting to be captured (last 10 minutes)
PUT /api/students/{id}/enrollment (+ /enrollment-image)	Report enrolled / failed and upload one JPEG
GET /api/devices/{id}/session	Who may enter now: seat list, already inside, session end
GET /api/devices/{id}/roster	Students enrolled on this device (delete other faces)
POST /api/access-events (+ /batch)	Report decisions; duplicates ignored by clientEventId

Appendix C: On-site accuracy test procedure

- Enrol at least 30 volunteers at the venue door under normal lighting; book them into one exam session.
- Genuine test: each volunteer walks up 5 times (150 attempts). Count GRANTED (correct), DENIED and wrong names.
- Impostor test: 20 people who are not enrolled walk up 5 times each (100 attempts). Count every GRANTED as a false accept.
- Repeat with strong light behind the student (window or open door) and in dim light.
- Report FAR = false accepts ÷ impostor attempts and FRR = denied genuine attempts ÷ genuine attempts, with the time per entry.

Appendix D: Benchmark method

Scripts: `embed_all.py` embeds every LFW image with `pc_device/face_engine.py` (models checked against the SHA-256 values in `download_models.py`); `analyse.py` draws the 1:1 pairs and runs the 1:N simulation with a fixed random seed (2026); `timing640.py` times the full pipeline on 640×480 frames. Environment: OpenCV 5.0.0, Python 3, Intel Xeon 2.1 GHz (4 cores). Images: a public mirror of the official LFW set (13 233 images, 5 749 people). The scripts are not part of the source repository and are available on request.

Appendix E: Source code

The full source code is in the Git repository github.com/TebogoClifford/Smart_Registration_System: `Backend/` (REST API, database migrations, tests), `web_app/` (SRS Console), `pc_device/` (PC door program and Arduino sketch) and `device_client/` (Python client library and simulator).